

Cloud AI & industrialisation

Industrialiser un code python



Python et industrialisation

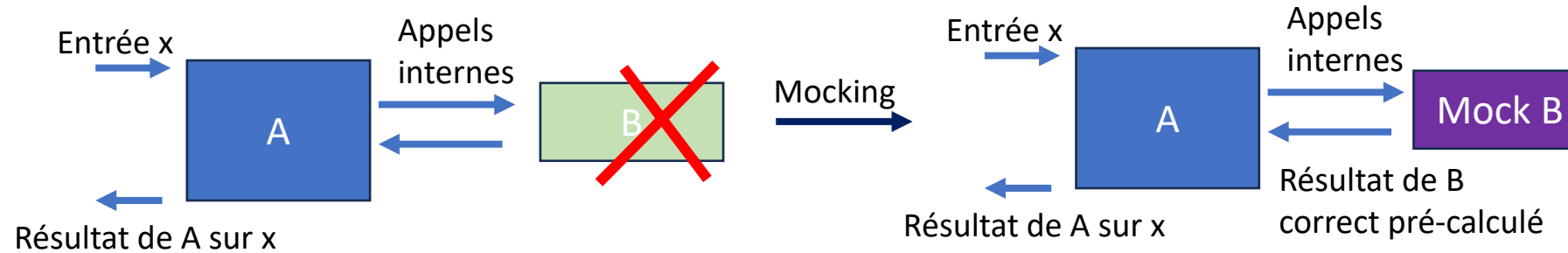
Pouvoir déployer en production du code nécessite du code « *de qualité industrielle* » :

- **Fiable** :
 - Sans bug, stable
 - Testé
- **Performant**
 - Testé par rapports aux performances et charges demandées
- **Maintenable** :
 - “un système doit être adapté ou il devient de moins en moins satisfaisant.” 1ère loi de Lehman.
 - Le code doit permettre une maintenance, des corrections et des évolutions faciles
 - Documenté
 - Bien structuré
 - Propre
- **Bonus: Réutilisable/Mutualisable**
 - *Le code est pensé pour être facilement utilisable dans d'autres applications*
 - *Critère de succès pour l'open-source (ex: scikit-learn)*

Un code fiable est un code testé

Les tests unitaires :

- Teste une fonctionnalité pour une méthode, indépendamment du reste du code.
- Simule (« **mock** ») le comportement des autres fonctions appelées par la méthode testée.



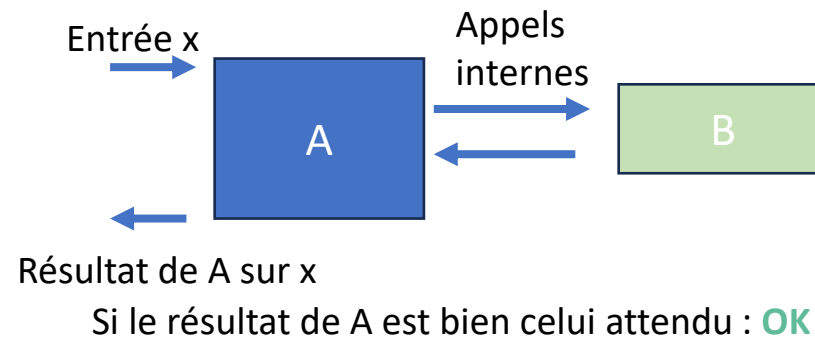
Si le résultat de A est bien celui attendu : **OK**

- Permet de ne tester que A

Un code fiable...

1. Les tests unitaires
2. Les tests d'intégration

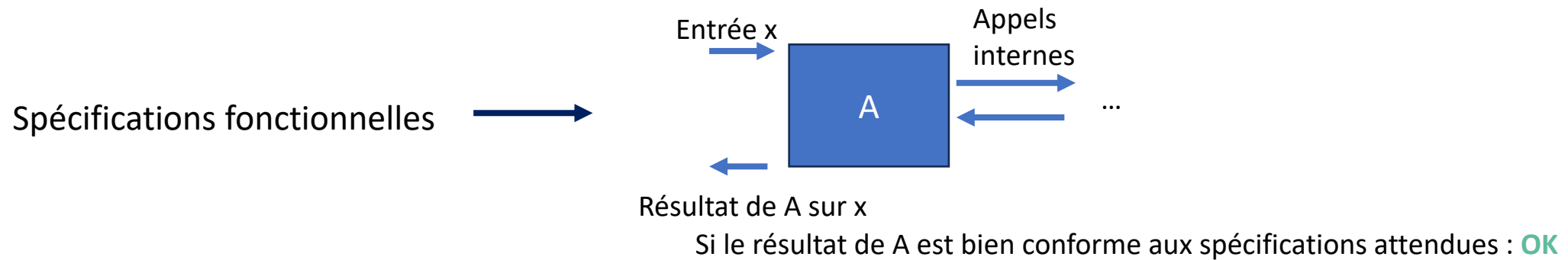
- Vérifient l'intégration de plusieurs composants
- Ne regarde que l'entrée et la sortie, pas l'appel des composants entre eux



- Permet de tester que A et B ensembles ont bien le bon fonctionnement

Un code fiable...

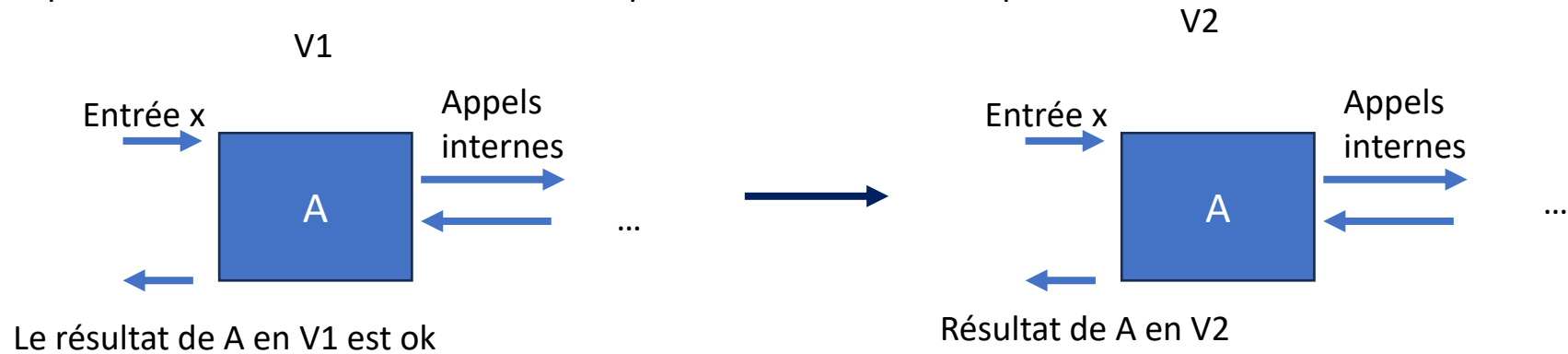
1. Les tests unitaires
2. Les tests d'intégration
3. Test fonctionnels
 - Similaires aux tests d'intégration
 - Vérifient une par une que chaque fonctionnalité désirée du programme est bien conforme à l'attendu



Un code fiable...

1. Les tests unitaires
2. Les tests d'intégration
3. Test fonctionnels
4. Test de non regression

- « Enregistre » un comportement qui fonctionne pour une version donnée du logiciel
- Vérifie que dans les versions futures, ce comportement reste identique.



Si le résultat de A en V2 est bien identique à celui de A en V1: **OK**

Un code fiable...

1. Les tests unitaires
 2. Les tests d'intégration
 3. Test fonctionnels
 4. Test de non-régression
 5. Test de performances
- Ceux que l'on fait en ML : on vérifie des performances sur des ensemble de données.
 - En termes de **précision à la tâche, mais aussi en termes de temps, délai de réponse, ...**
 - **Test de charge** : on test en (sur)-chargeant le système
 - On définit différents scénarios d'utilisation du système
 - Ex: max 100 utilisateurs simultanés, max 10 000 requêtes/min, etc, ...
 - On test avec des charges égales ou supérieures aux estimations
 - Pour permettre une marge d'erreur

Un code fiable...

1. Les tests unitaires
2. Les tests d'intégration
3. Test fonctionnels
4. Test de non-régression
5. Test de performances

En python :

- **pytest** ou **unittest** pour les tests.
- ***unittest.mock***, ***patch*** ou ***magickmock*** sont souvent utilisés pour mocker une fonction ou méthode dans le cadre de tests unitaires ou d'intégration partiels.

Un code performant

- Nous avons déjà parlé de code performant en python
 - Numpy, MKL, vectorisation, calculs sur gpus plutôt que cpu, etc...
 - Suivre les bonnes pratiques.
 - Les tests de charge peuvent nous permettre d'optimiser des paramètres tels que le batch, les ressources cpu/gpu/mémoire nécessaires.
- Vous pouvez utiliser **cProfile** (+**SnakeViz**) pour voir quelles parties de votre code sont les plus appelées, ou quelles sont celles dans lesquelles on passe le plus de temps
 - **Permet de savoir quoi optimiser dans le code**

Un code maintenable

Il respecte le même standard de code pour l'ensemble du projet.

- Ex: pep8.
 - Règle de convention de nommage
 - minuscules et underscore entre les termes partout
 - `ma_variable`, `mon_instance_de_classe`, etc...
 - Sauf :
 - Variable globale : tout en majuscule: `MA_VARIABLE_GLOBALE`
 - Nom de classe : en CamelCase. Ex: `MaClasseDeTest`
 - Underscore pour indiquer tous ce qui est privé (variables, nom de fichiers, etc).
 - Règle de mise en forme des fichiers
 - Règles de bonnes pratiques
 - La seule modification que l'on fait: le nombre de caractères par lignes !
- Utiliser des linters pour vérifier la qualité du code
 - Ex : **pylint** : outil permettant de noter la qualité du code sur 10
 - `pip install pylint`
 - `pylint your_module`
 - A des limites, comme tout outil automatique

Un code maintenable

Il est documenté :

- Il possède un Readme, une documentation d'installation et d'utilisation
 - Comment installer la solution
 - En python, demande souvent un requirement.txt (pip), conda.yaml (conda), env.tar.gz (conda-pack), etc...
 - Même si docker est utilisé
 - Comment utiliser la solution
 - En python, les notebooks sont un format très adapté pour montrer comment utiliser la solution
- Il possède une documentation technique
 - Ex: Documentation Sphinx et google markdown
 - Permet de générer une doc html à partir de commentaires dans le code
 - A de nombreuses fonctionnalités plus avancées, mais qui demandent un peu de temps à mettre en place
 - Intégrer des notebooks d'exemple d'utilisation du code dans la documentation
 - Intégrer des fichiers pdf dans la documentation

Un code maintenable

Il respecte des principes de conceptions:

- **KISS:** Keep it simple, stupid: choisir la façon la plus simple.
 - Particulièrement au niveau des interfaces
- **YAGNI:** You ain't gonna need it, ou principe « **d'éviter les cables débranchés** »
 - Ne pas éviter d'avoir des cables débranchés partout évite de se prendre les pieds dedans.
 - Ne pas chercher à gérer des cas qui pourraient ou pourraient ne pas arriver.
 - Ne pas maintenir du code qui pourrait ou pourrait ne pas servir dans le futur.
- **Moindre surprise:** le comportement doit être celui auquel on s'attendrait.
- **Compartmentaliser :**
 - « on ne parle pas aux étrangers »
 - on ne doit pas savoir ce qu'il se passe dans les entités/objets avec lesquelles on interagit.

Un code maintenable

En python :

- Il est structuré **sous forme de modules et sous modules**
 - Permet de compartimenter le code
 - La visibilité en dehors du module est déclarée dans le `__init__.py` du module
 - Permet de s'assurer que le code pourra être facilement packagé, utilisé dans un système tiers
- Vos notebooks et scripts **utilisent votre module**
 - Si vos notebooks et scripts sont simples à écrire, lire, et ont peu de code, c'est généralement que votre module est bien fait
 - En général, cela veut dire que son intégration en production sera elle-aussi facile
- KISS: ce n'est pas parce que python permet des fonctionnalités complexes que celles-ci sont de bonnes idées
 - Exemple que j'ai pu constater en ne le respectant pas au début
 - Exemple que devraient suivre plusieurs industriels dont j'ai pu voir le code
- Il utilise comme interface des objets de la bibliothèque standard ou commun (Kiss, moindre surprise).
 - Exemple que ne suivaient pas plusieurs industriels pour lesquels on a intégré leur code

Un exemple

Scikit-learn

- <https://github.com/scikit-learn/scikit-learn>
- Doc
- Modules, init.py
- .gitignore
- Examples
- Interfaces avec des objets standards
-